Universidad Tecnológica de Campeche

Ingeniería en Desarrollo y Gestión de Software

Actividad 2 · Segunda parte

Pruebas de software e integración continua del sistema de control de inventario

Asignatura: Gestión del Proceso de Desarrollo de Software

Enfoque: Pruebas de software e integración continua con metodología DevOps

Tipo de trabajo: Documento técnico de análisis, diseño y validación de software

Equipo

Emily Yamilett Pinzón Sánchez · Gestora de proyectos

Chuina Guadalupe García Campos · Analista y Tester

Pablo Alejandro Magaña Hernández · Desarrollador / DevOps

Emmanuel Hernández Barbis · Diseñador

Ana Edith Rodríguez Pérez · Cliente (aceptación)

San Francisco de Campeche, Campeche · 30 de octubre de 2026

Introducción

Una tienda necesita saber qué mercancía tiene, cuánto le queda y qué salió con cada venta. Si la existencia registrada no coincide con la real, se ofrecen artículos agotados o no se piden a tiempo los que están por acabarse. El caso de estudio es un sistema de control de inventario que registra productos, consulta existencias y actualiza las cantidades al vender.

Trabajamos sobre el sistema que el equipo ya tiene (PHP 8, API JSON y archivo de datos JSON). Escribimos cinco requisitos funcionales, el plan de pruebas, diez casos clasificados por nivel, la bitácora con el porcentaje de aprobación, dos defectos documentados, la propuesta para repetir las pruebas al corregirlos y el flujo de integración continua en GitHub Actions.

Seguimos un enfoque DevOps: las pruebas están automatizadas con PHPUnit y se ejecutan cada vez que alguien sube un cambio al repositorio, así un error se detecta antes de integrarse y no cuando el sistema ya está en uso. Los resultados que presentamos provienen de ejecuciones reales, no de datos simulados.

- **Código y pruebas:** Actividad 2/sistema (src, tests y scripts)
- **Evidencia:** sistema/evidencias (JUnit, reporte HTML, JSON y salida de consola)
- **Flujo de CI:** sistema/.github/workflows/ci.yml
- **Diagramas editables:** Diagramas - Segunda parte.pptx (niveles, matriz RACI, trazabilidad, gráfica de aprobación, ciclo de re-pruebas y flujos de CI)
- **Repositorio en GitHub:** https://github.com/azharsonix/sistema-inventario-actividad-2

1. Identificación de requisitos funcionales

Los requisitos funcionales indican las operaciones que el sistema debe realizar para cubrir las necesidades de sus usuarios. Cada uno lleva un criterio de aceptación que se puede comprobar con una prueba.

ID	Req. funcional	Qué hace el sistema	Criterio de aceptación
RF-01	Registrar productos	Captura código, nombre, categoría, precio, existencia inicial y stock mínimo, y valida cada dato.	El producto se guarda solo si los datos son válidos y el código no está repetido. Rechaza nombre vacío, precio negativo, existencia negativa y códigos fuera de formato (3 a 20 caracteres A-Z, 0-9 o guion).
RF-02	Consultar existencias	Busca un producto por su código (sin importar mayúsculas o espacios) y lista la existencia de todos.	Muestra los datos del producto correcto o el mensaje «No existe un producto con el código …».
RF-03	Registrar ventas y actualizar la existencia	Registra la salida por venta, descuenta las unidades vendidas y guarda el movimiento con la existencia resultante.	Existencia final = existencia inicial − cantidad vendida. Rechaza cantidades de cero, negativas o mayores a la existencia sin cambiar ningún dato.
RF-04	Registrar entradas de mercancía	Suma a la existencia la cantidad recibida del proveedor.	La existencia aumenta en la cantidad recibida y la entrada aparece en el historial de movimientos.
RF-05	Reporte de inventario y resurtido	Calcula total de productos, total de unidades, valor del inventario y productos por resurtir.	Lista para resurtir todo producto con existencia igual o menor a su stock mínimo y los totales coinciden con los datos.

Relación entre requisitos y pruebas: cada requisito tiene al menos un caso de prueba que verifica su cumplimiento; a esta relación se le llama trazabilidad. La matriz indica qué casos cubren cada requisito: todos tienen dos o más.

RF	CP 01	CP 02	CP 03	CP 04	CP 05	CP 06	CP 07	CP 08	CP 09	CP 10
RF-01	✓	✓					✓			
RF-02					✓		✓			✓
RF-03			✓	✓	✓	✓	✓	✓		
RF-04										✓
RF-05									✓	

2. Plan de pruebas de software

2.1 Objetivo

Comprobar que el sistema registra productos, consulta existencias, valida los datos que recibe y actualiza las cantidades con cada venta y cada entrada según RF-01 a RF-05, sin dejar el inventario en un estado incorrecto cuando rechaza una operación.

2.2 Alcance

El plan de pruebas incluye:

- Registro de productos nuevos y rechazo de códigos duplicados.
- Validación de datos: nombre requerido, precio y cantidades no negativos, formato del código.
- Consulta de productos existentes e inexistentes.
- Ventas con existencia suficiente e insuficiente.
- Actualización de la existencia con cada venta y cada entrada de mercancía.
- Persistencia de los datos en el archivo JSON.
- Reporte de inventario con productos por resurtir.
- Ejecución automática de las pruebas con integración continua (GitHub Actions).

Quedan fuera del alcance la interfaz gráfica, usuarios y permisos, facturación fiscal, pagos electrónicos, contabilidad, administración de proveedores y pruebas de carga.

2.3 Recursos necesarios

Recurso	Qué usamos
Hardware	Laptop del equipo con Windows 11 Pro y máquinas virtuales ubuntu-latest de GitHub Actions.
Software	Sistema de inventario (PHP 8.4.27), archivo de datos JSON y servidor integrado de PHP (php -S) para las pruebas por HTTP.
Control de versiones	Git y repositorio público en GitHub con ramas main, develop, feature/** y fix/**.
Pruebas automatizadas	PHPUnit 11.5.56 con Composer; cuatro suites: Unitarias, Integracion, Sistema y Aceptacion.
Integración continua (CI)	GitHub Actions con el flujo .github/workflows/ci.yml en PHP 8.2, 8.3 y 8.4.
Datos de prueba	Productos ficticios (LAP-001, MOU-010, TEC-005, ACC-002, RED-001) con distintos precios, existencias y mínimos. Cada prueba crea su archivo JSON temporal y lo borra cuando acaba.
Documentación	Plan de pruebas, casos, bitácora de ejecución, informe de defectos y reporte HTML generado por scripts/ejecutar-pruebas.php.

2.4 Responsables

Rol	Integrante	Responsabilidades
Analista de requisitos	Chuina Guadalupe García Campos	Redacta y valida los requisitos funcionales con la cliente.
Desarrollador	Pablo Alejandro Magaña Hernández	Implementa el sistema y corrige los defectos.
Ingeniera de pruebas (QA)	Chuina Guadalupe García Campos	Diseña, ejecuta y documenta las pruebas; registra los defectos.
Responsable DevOps	Pablo Alejandro Magaña Hernández	Configura el repositorio, la protección de ramas y el flujo de integración continua.
Product Owner / usuaria representante	Ana Edith Rodríguez Pérez	Valida que el sistema cumpla lo que necesita la tienda (CP-09 y CP-10).
Gestora de proyectos	Emily Yamilett Pinzón Sánchez	Aprueba el plan, revisa los Pull Requests y da seguimiento a los defectos.
Diseñador	Emmanuel Hernández Barbis	Revisa que los mensajes de error sean claros para el usuario.

Nota: somos un equipo pequeño y algunas personas cumplen más de un rol. Para conservar la revisión, la persona que corrige un defecto no lo verifica: Pablo hace la corrección y Chuina repite las pruebas.

Matriz RACI

La matriz indica, para cada actividad de pruebas, quién la realiza (R), quién la aprueba y rinde cuentas (A), a quién se consulta antes de decidir (C) y a quién se informa del resultado (I). Cada actividad tiene una sola A.

Actividad	Emily Gestora	Chuina Analista/QA	Pablo Dev/DevOps	Emmanuel Diseñador	Ana Cliente
Redactar los requisitos funcionales	A	R	C	I	C
Elaborar el plan de pruebas	A	R	C	I	I
Diseñar los casos de prueba	A	R	C	I	C
Automatizar las pruebas	A	C	R	I	I
Ejecutar las pruebas y registrar resultados	A	R	I	I	I
Registrar y priorizar defectos	A	R	C	C	I
Corregir defectos	A	C	R	I	I
Repetir pruebas y regresión	A	R	C	I	I
Configurar el CI y la protección de ramas	A	I	R	I	I
Revisar los mensajes de error	A	C	C	R	I
Validar los escenarios de aceptación	I	R	I	I	A
Aprobar el Pull Request e integrar	R/A	C	C	I	I

R = Responsable · A = Aprueba · C = Consultado · I = Informado. Versión editable: Diagramas - Segunda parte.pptx, diapositiva 2.

2.5 Criterios de entrada y salida

Criterios de entrada:

- Los requisitos RF-01 a RF-05 están documentados.
- Existe una versión ejecutable del sistema en la rama y composer install termina sin errores.
- El entorno de pruebas está configurado (PHP, PHPUnit y el servidor de pruebas levantan).
- Los datos de prueba están preparados en los archivos de prueba.

Criterios de salida:

- Los diez casos planificados se ejecutaron.
- Los resultados están en la bitácora y en sistema/evidencias.
- No queda ningún defecto crítico o de prioridad alta abierto.
- Cada defecto corregido tiene su prueba de repetición aprobada.
- La regresión completa y el pipeline de CI están en verde.

3. Diseño de casos de prueba

Diseñamos diez casos (la práctica pide al menos seis). Cada uno está automatizado y lleva su ID en el nombre de la prueba, por lo que el reporte y GitHub Actions muestran la misma clave que este documento.

Caso de prueba CP-01: Registro correcto de producto

Caso	CP-01 – Registro válido (unitaria)
Requisito asociado	RF-01
Objetivo	Comprobar que un producto con datos válidos se registra.
Precondición	El inventario está vacío y el código MOU-010 no existe.
Datos de entrada	Código: « mou-010 » (con espacios y en minúsculas) Nombre: Mouse inalámbrico · Categoría: Periféricos Precio: $250.00 · Existencia inicial: 30 · Stock mínimo: 5
Procedimiento	1. Crear un inventario vacío. 2. Registrar el producto con los datos de entrada. 3. Consultar el producto por su código.
Resultado esperado	Se guarda con código MOU-010 y existencia 30; el inventario tiene 1 producto.
Prueba automatizada	tests/Unitarias/RegistroProductoTest.php

Caso de prueba CP-02: Rechazo de código duplicado

Caso	CP-02 – Validación de datos (unitaria)
Requisito asociado	RF-01
Objetivo	Comprobar que el sistema no permite dos productos con el mismo código.
Precondición	Existe LAP-001 (Laptop Lenovo).
Datos de entrada	Código: «lap-001» · Nombre: Otra laptop
Procedimiento	1. Intentar registrar el producto. 2. Revisar el mensaje. 3. Contar los productos y revisar el nombre de LAP-001.
Resultado esperado	Error «Ya existe un producto con el código LAP-001.»; sigue habiendo 1 producto y conserva su nombre.
Prueba automatizada	tests/Unitarias/RegistroProductoTest.php

Caso de prueba CP-03: Venta con existencia suficiente

Caso	CP-03 – Venta válida (unitaria)
Requisito asociado	RF-03
Objetivo	Comprobar que una venta descuenta la cantidad vendida y guarda el movimiento.
Precondición	LAP-001 con existencia 10.
Datos de entrada	Venta de 4 piezas de LAP-001.
Procedimiento	1. Registrar la salida por venta. 2. Consultar la existencia. 3. Revisar el último movimiento.
Resultado esperado	Existencia 6. Movimiento de salida con cantidad 4, existencia resultante 6 y motivo «Venta».
Prueba automatizada	tests/Unitarias/VentaTest.php

Caso de prueba CP-04: Venta con existencia insuficiente

Caso	CP-04 – Inventario insuficiente (unitaria)
Requisito asociado	RF-03
Objetivo	Comprobar que el sistema rechaza vender más de lo que hay.
Precondición	LAP-001 con existencia 10.
Datos de entrada	Venta de 50 piezas de LAP-001.
Procedimiento	1. Intentar registrar la venta. 2. Revisar el mensaje. 3. Consultar existencia y movimientos.
Resultado esperado	Error «Existencia insuficiente de LAP-001…»; la existencia sigue en 10 y no se registra movimiento.
Prueba automatizada	tests/Unitarias/VentaTest.php

Caso de prueba CP-05: Persistencia de una venta

Caso	CP-05 – Venta guardada (integración)
Requisito asociado	RF-02, RF-03
Objetivo	Comprobar que la venta queda guardada en el archivo y se lee igual al abrirlo otra vez.
Precondición	Archivo JSON con LAP-001 (10) y MOU-010 (30).
Datos de entrada	Venta de 7 piezas de MOU-010.
Procedimiento	1. Registrar la venta con el repositorio. 2. Abrir el mismo archivo con un repositorio nuevo. 3. Consultar existencias y movimientos.
Resultado esperado	LAP-001: 10 y MOU-010: 23; un movimiento con existencia resultante 23.
Prueba automatizada	tests/Integracion/VentaPersistidaTest.php

Caso de prueba CP-06: Venta rechazada no modifica el archivo

Caso	CP-06 – Consistencia de datos (integración)
Requisito asociado	RF-03
Objetivo	Comprobar que una venta rechazada no escribe nada en el archivo de datos.
Precondición	El mismo archivo JSON del CP-05.
Datos de entrada	Venta de 11 piezas de LAP-001.
Procedimiento	1. Guardar una copia del archivo. 2. Intentar la venta dentro de una actualización del repositorio. 3. Comparar el archivo con la copia.
Resultado esperado	La venta se rechaza, el archivo queda idéntico y LAP-001 sigue en 10.
Prueba automatizada	tests/Integracion/VentaPersistidaTest.php

Caso de prueba CP-07: Alta, consulta y venta por la API

Caso	CP-07 – Flujo por HTTP (sistema)
Requisito asociado	RF-01, RF-02, RF-03
Objetivo	Comprobar que el sistema completo responde correctamente por HTTP.
Precondición	Servidor de pruebas en ejecución con LAP-001.
Datos de entrada	TEC-005, 14 piezas; venta de 3 piezas.
Procedimiento	1. POST /api/productos con TEC-005. 2. GET /api/productos/tec-005. 3. POST /api/productos/TEC-005/salidas con cantidad 3. 4. GET del producto.
Resultado esperado	Alta 201; consulta con existencia 14; venta 201 con existencia 11 y motivo «Venta»; consulta final con 11.
Prueba automatizada	tests/Sistema/FlujoVentaHttpTest.php

Caso de prueba CP-08: Rechazo de cantidades inválidas por la API

Caso	CP-08 – Validación por HTTP (sistema)
Requisito asociado	RF-03
Objetivo	Comprobar que la API rechaza cantidades de cero, negativas o mayores a la existencia.
Precondición	Servidor de pruebas con LAP-001 (10).
Datos de entrada	Cantidades 0, −2 y 11.
Procedimiento	1. Enviar las tres ventas a POST /api/productos/LAP-001/salidas. 2. Consultar LAP-001 y el historial.
Resultado esperado	Las tres responden 422 con mensaje de error; LAP-001 sigue en 10 y el historial está vacío.
Prueba automatizada	tests/Sistema/FlujoVentaHttpTest.php

Caso de prueba CP-09: Cierre del día con resurtido

Caso	CP-09 – Escenario de la cliente (aceptación)
Requisito asociado	RF-05
Objetivo	Comprobar que el reporte le indica a la cliente qué pedir al cierre del día.
Precondición	ACC-002 (12, mín. 5), MOU-010 (30, mín. 10), RED-001 (6, mín. 4).
Datos de entrada	Ventas: 7 ACC-002, 2 RED-001 y 5 MOU-010.
Procedimiento	1. Registrar las ventas del día. 2. Abrir /api/reporte.
Resultado esperado	Por resurtir: ACC-002 y RED-001 (no MOU-010). 3 productos, 34 unidades, valor $10,291.00.
Prueba automatizada	tests/Aceptacion/EscenariosTiendaTest.php

Caso de prueba CP-10: Llegada de mercancía y venta en mostrador

Caso	CP-10 – Escenario de la cliente (aceptación)
Requisito asociado	RF-02, RF-04
Objetivo	Comprobar que existencias e historial cuadran con entradas y ventas.
Precondición	Los mismos tres productos del CP-09.
Datos de entrada	Entrada de 10 MOU-010; venta de 3 MOU-010.
Procedimiento	1. Registrar la entrada. 2. Registrar la venta. 3. Consultar existencias e historial.
Resultado esperado	MOU-010: 37 (ACC-002: 12, RED-001: 6). Historial: salida de 3 → 37 y entrada de 10 → 40.
Prueba automatizada	tests/Aceptacion/EscenariosTiendaTest.php

4. Clasificación de los casos de prueba

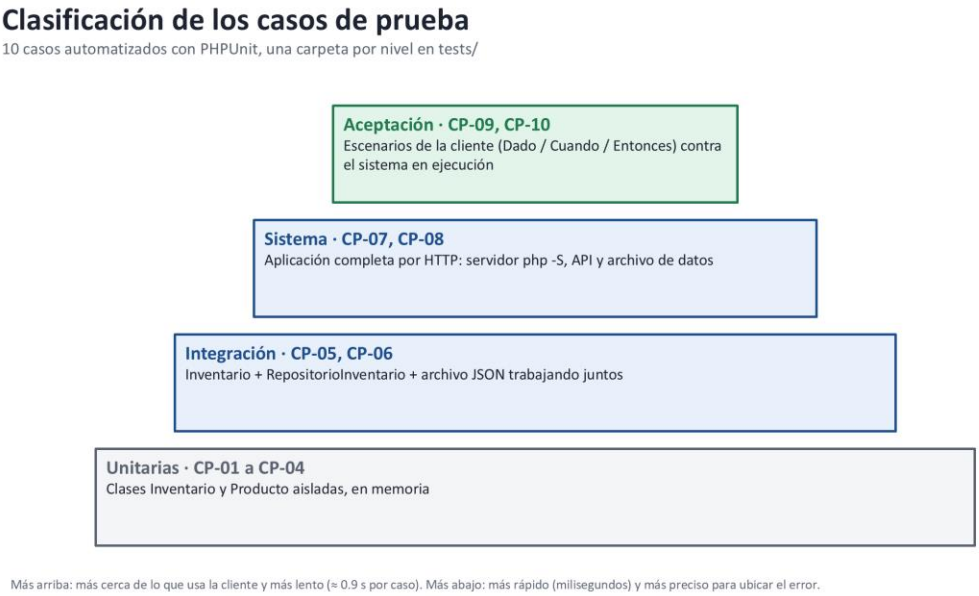


Figura 1. Niveles de prueba y casos de cada uno.

Caso	Tipo de prueba	Justificación
CP-01	Unitaria	Revisa la función de registro de la clase Inventario en memoria, sin archivo ni servidor.
CP-02	Unitaria	Revisa una regla de validación (código único) dentro de una sola clase.
CP-03	Unitaria	Revisa el cálculo de la existencia al vender en la clase Inventario.
CP-04	Unitaria	Revisa el rechazo por existencia insuficiente en la misma clase.
CP-05	Integración	Junta Inventario, RepositorioInventario y el archivo JSON: la venta se guarda y se vuelve a leer.
CP-06	Integración	Comprueba que las clases y el archivo no quedan a medias cuando una venta se rechaza.
CP-07	Sistema	Prueba la aplicación completa por HTTP: rutas, respuestas 201 y datos guardados.
CP-08	Sistema	Prueba la validación de la API completa con códigos 422.
CP-09	Aceptación	Escenario real de la cliente (cierre del día); Ana Edith revisa y aprueba el resultado.
CP-10	Aceptación	Escenario real de la cliente (llegada de mercancía y venta en mostrador).

5. Ejecución de las pruebas y registro de resultados

Registramos la ejecución en una bitácora de pruebas. Las cuatro suites se ejecutaron con scripts/ejecutar-pruebas.php el 30 de octubre de 2026 (Windows NT 10.0, PHP 8.4.27, PHPUnit 11.5.56, versión 185cafb del repositorio).

```
cd "Actividad 2\sistema"
composer evidencia     # ronda 1: los 10 casos con el código actual
composer regresion     # rondas 2 a 4: DEF-01 y DEF-02 inyectados y código corregido
```

5.1 Bitácora de ejecución

Ronda 1 · código actual:

ID	Resultado esperado	Resultado obtenido	Estado	Tiempo

ID	Resultado esperado	Resultado obtenido	Estado	Tiempo
CP-01	Producto registrado correctamente	Producto MOU-010 registrado con existencia 30; inventario con 1 producto.	Aprobado	0.008 s
CP-02	Código duplicado rechazado	«Ya existe un producto con el código LAP-001.»; 1 producto, nombre Laptop Lenovo.	Aprobado	0.001 s
CP-03	Existencia descontada	Existencia 6; movimiento de salida de 4 piezas, resultante 6, motivo Venta.	Aprobado	0.001 s
CP-04	Venta rechazada sin cambios	«Existencia insuficiente de LAP-001: hay 10 y se pidieron 50.»; existencia 10; sin movimientos.	Aprobado	0.001 s
CP-05	Venta guardada en el archivo	Al abrir el archivo otra vez: LAP-001 10, MOU-010 23; 1 movimiento con resultante 23.	Aprobado	0.009 s
CP-06	Archivo sin cambios	Venta rechazada; el archivo no cambió; LAP-001 en 10.	Aprobado	0.002 s
CP-07	Flujo completo por HTTP	201 → existencia 14 → 201 con existencia 11 y motivo Venta → existencia 11.	Aprobado	0.889 s
CP-08	Tres respuestas 422	422, 422 y 422 con mensaje; LAP-001 en 10; historial vacío.	Aprobado	0.865 s
CP-09	Reporte con productos a resurtir	Por resurtir: ACC-002 y RED-001; 3 productos, 34 unidades, $10,291.00.	Aprobado	0.880 s
CP-10	Existencias e historial cuadran	MOU-010 37, ACC-002 12, RED-001 6; historial: salida 3 → 37, entrada 10 → 40.	Aprobado	0.872 s

Rondas 2 a 4 · regresión con defectos inyectados:

El script scripts/simular-defectos.php introduce DEF-01 y DEF-02 (sección 6) en una copia de src/, ejecuta los diez casos sobre esa copia y repite la ronda con el código corregido. Los archivos originales no se modifican.

Ronda	Casos fallidos	Resultado obtenido en los casos fallidos	Estado
2 · con DEF-01	CP-04, CP-06, CP-08	CP-04: Se esperaba InventarioException por existencia insuficiente. CP-06: Se esperaba que la venta se rechazara. CP-08: La venta de 11 unidades de LAP-001 (existencia 10) debía responder 422.	Fallido (7 de 10)
3 · con DEF-02	CP-09	CP-09: Productos a resurtir: se esperaban ACC-002 (5 de mínimo 5) y RED-001 (4 de mínimo 4).	Fallido (9 de 10)
4 · corregido	Ninguno	Los diez casos pasan.	Aprobado (10 de 10)

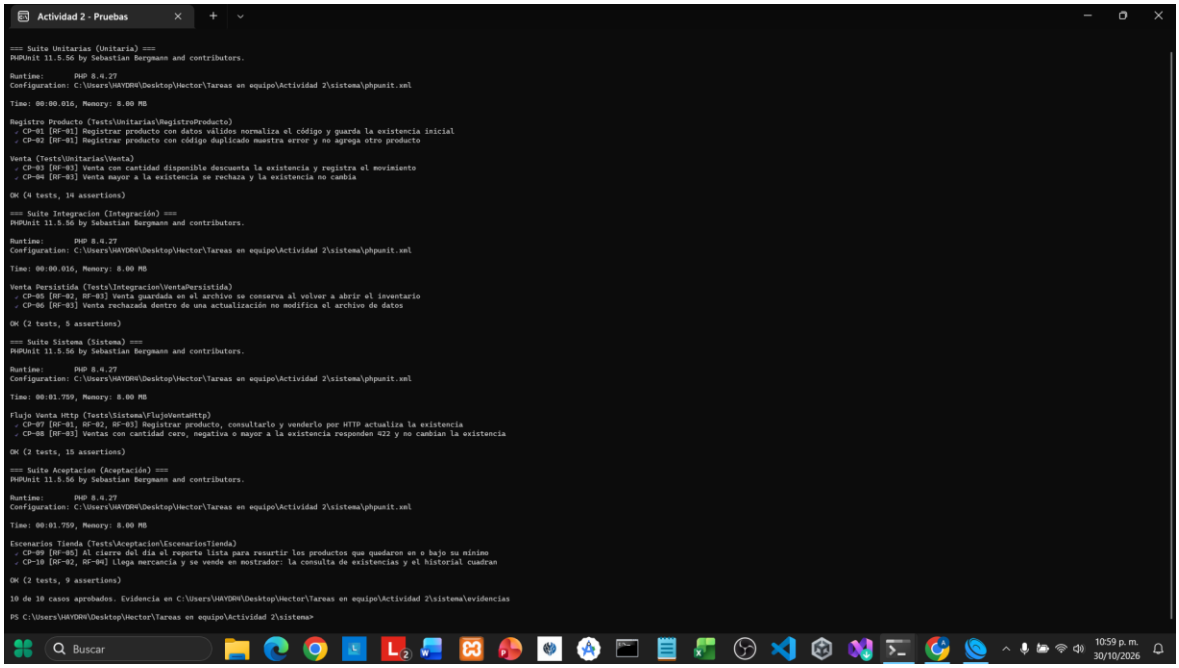


Figura 2. Ejecución de la ronda 1 con composer evidencia (30 de octubre de 2026).

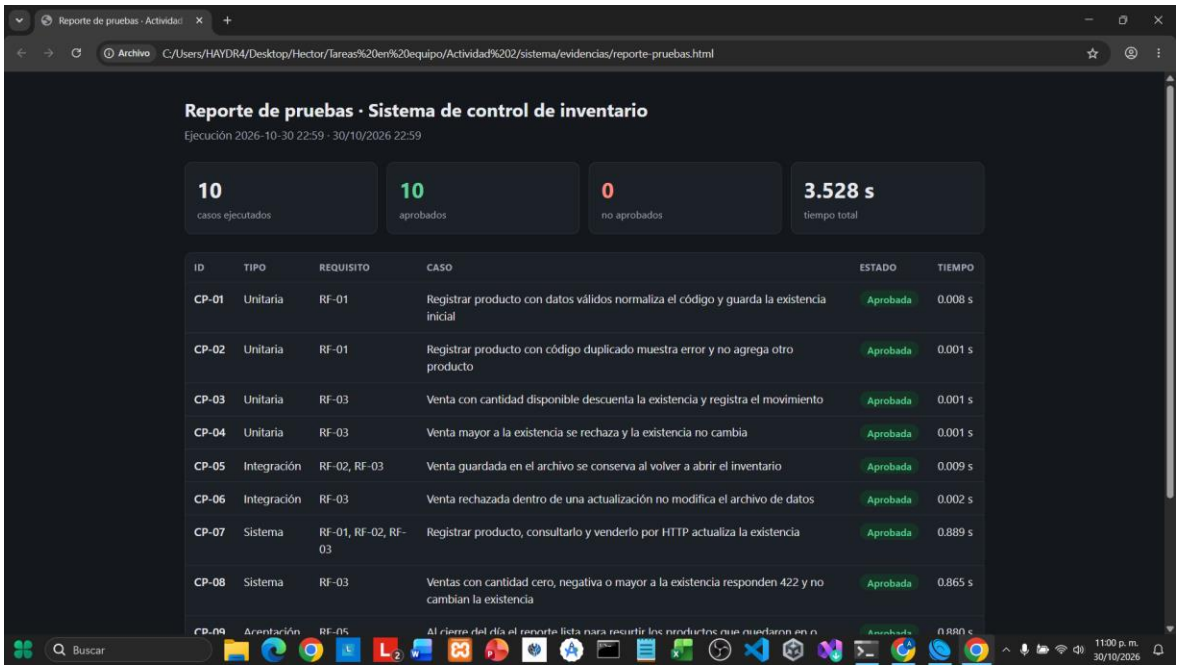


Figura 3. Reporte de la ronda 1 generado en evidencias/reporte-pruebas.html.

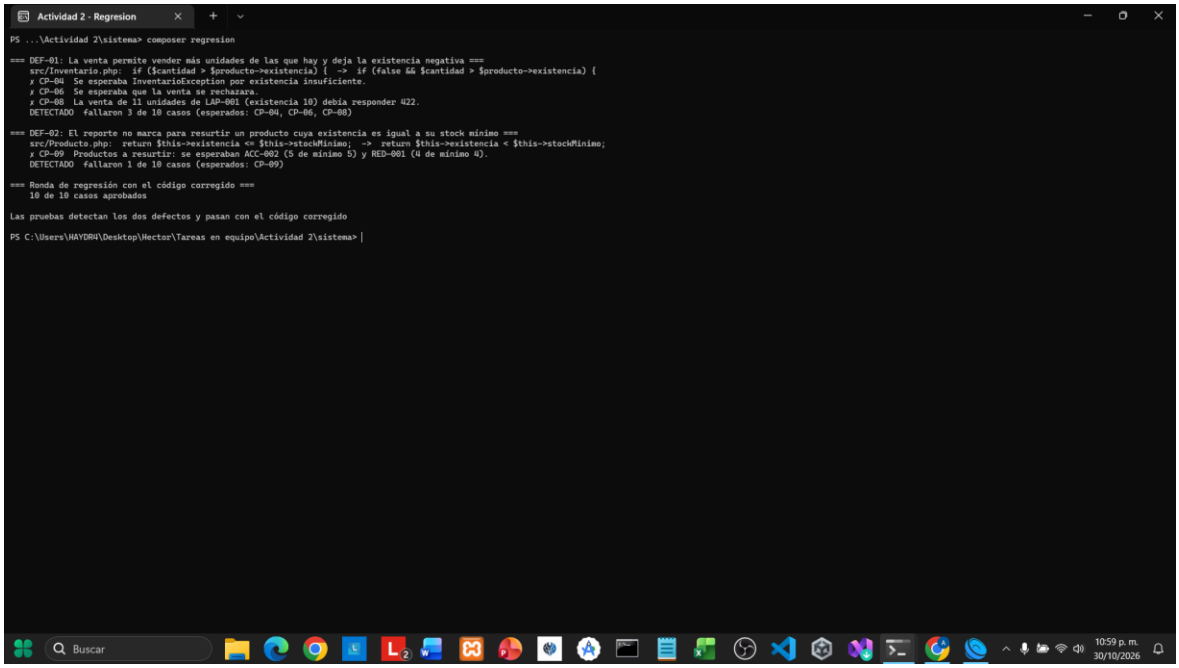


Figura 4. Rondas 2 a 4 con composer regresion: DEF-01 y DEF-02 inyectados y código corregido.

5.2 Cálculo del porcentaje de pruebas aprobadas

Porcentaje de aprobación = pruebas aprobadas / pruebas ejecutadas × 100

Ronda	Ejecutadas	Aprobadas	Fallidas	Cálculo	Aprobación
Ronda 1 · código actual	10	10	0	10 / 10 × 100	100 %
Ronda 2 · con DEF-01	10	7	3	7 / 10 × 100	70 %
Ronda 3 · con DEF-02	10	9	1	9 / 10 × 100	90 %
Ronda 4 · código corregido	10	10	0	10 / 10 × 100	100 %

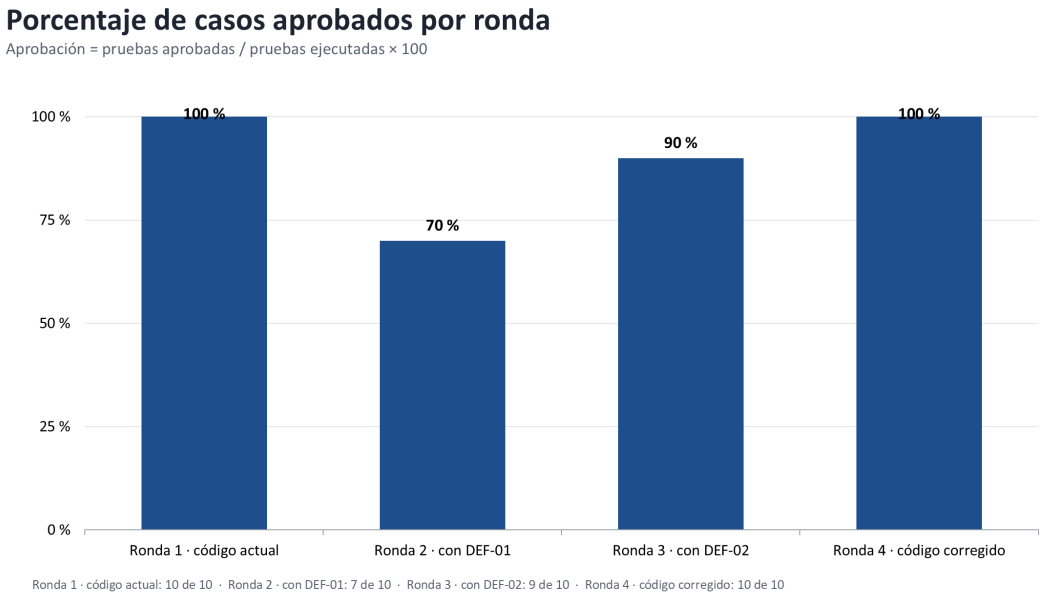


Figura 5. Porcentaje de casos aprobados en cada ronda.

Interpretación. Con el código actual pasan los diez casos (100 %) y se cumple el criterio de salida. Con DEF-01 la aprobación cae a 70 %: fallan CP-04, CP-06 y CP-08, que prueban la misma regla (no vender más de lo que hay) en tres niveles distintos. Con DEF-02 queda en 90 %, ya que solo CP-09 revisa el resurtido. En ambas rondas fallan justo las pruebas ligadas al requisito afectado, lo que confirma que detectan el error en el lugar correcto. Ese 90 % deja ver un riesgo: un porcentaje alto no basta para liberar, porque la prueba fallida corresponde a un defecto de prioridad media que provocaría faltantes en la tienda. Al corregir el código, la aprobación vuelve a 100 %.

5.3 Sistema en ejecución

Levantamos el sistema con php -S y lo usamos por HTTP con el producto del ejemplo de clase (TEC-020, Teclado USB, $250.00, 20 piezas). La Figura 6 muestra el alta, la consulta, una venta válida y tres operaciones que el sistema rechaza: vender 50 piezas cuando hay 16, vender una cantidad negativa y registrar un código repetido.

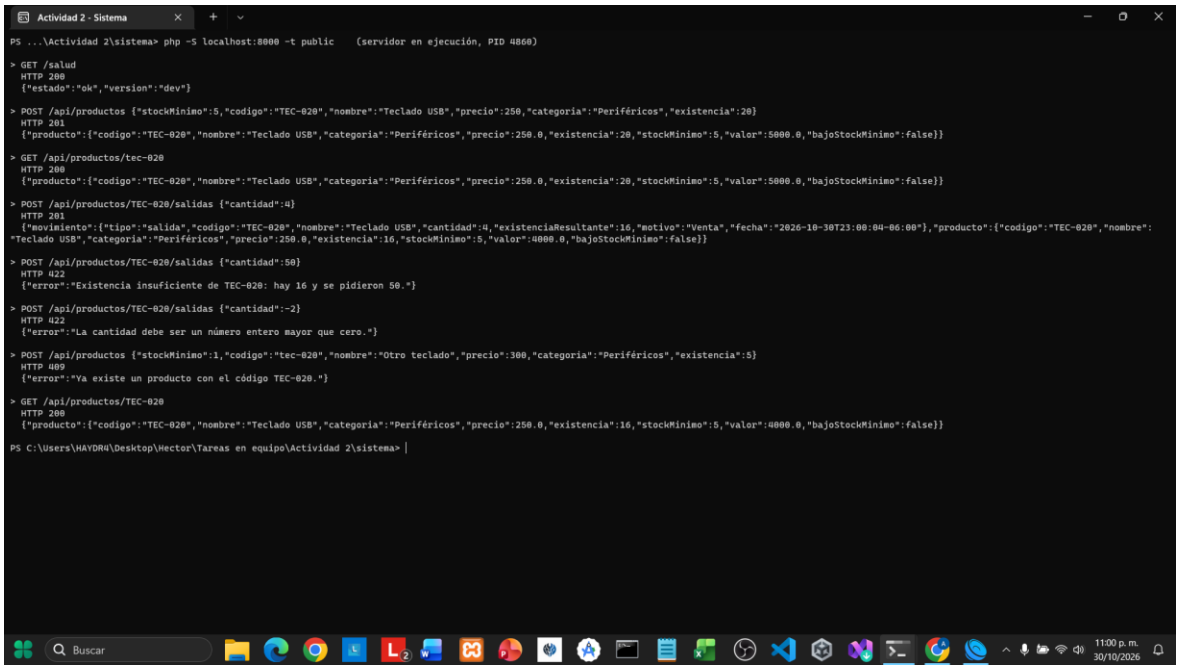


Figura 6. Peticiones al sistema en ejecución y sus respuestas (201, 422 y 409).

Operación	Respuesta	Caso relacionado
Alta de TEC-020 con 20 piezas	201 · producto registrado	CP-01, CP-07
Consulta de «tec-020» en minúsculas	200 · existencia 20	CP-07
Venta de 4 piezas	201 · existencia resultante 16	CP-03, CP-07
Venta de 50 piezas	422 · «Existencia insuficiente de TEC-020: hay 16 y se pidieron 50.»	CP-04, CP-08
Venta de −2 piezas	422 · «La cantidad debe ser un número entero mayor que cero.»	CP-08
Alta repetida de «tec-020»	409 · «Ya existe un producto con el código TEC-020.»	CP-02

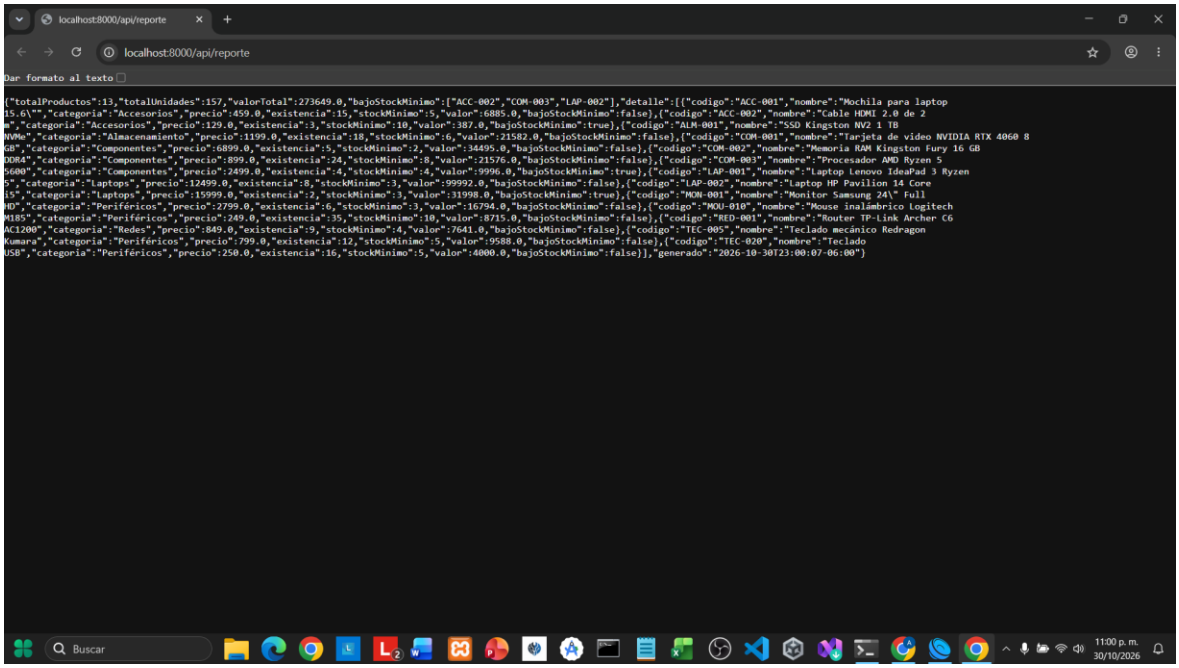


Figura 7. Reporte de inventario (/api/reporte) con los productos por resurtir.

6. Documentación de defectos

Documentamos dos defectos hipotéticos. Para contar con evidencia real y no solo esperada, los reprodujimos en una copia del código (rondas 2 y 3 de la sección 5.1).

Defecto DEF-01: la venta deja la existencia en negativo

DEF-01 · validación incorrecta de la venta (prioridad alta)

Descripción	Al registrar una venta mayor a la existencia disponible, el sistema la acepta, responde 201 y deja el inventario en negativo. La causa sería que Inventario::registrarSalida deja de comparar la cantidad pedida con lo disponible.
Requisito afectado	RF-03
Pasos para reproducir	1. Tener LAP-001 con existencia 10. 2. Enviar POST /api/productos/LAP-001/salidas con {"cantidad": 11}. 3. Consultar GET /api/productos/LAP-001.
Resultado esperado	El sistema rechaza la venta con 422 «Existencia insuficiente de LAP-001…» y la existencia sigue en 10.
Resultado obtenido	Responde 201, la existencia queda en −1 y se guarda un movimiento con existencia resultante −1.
Impacto	Se venden productos que no hay y el inventario deja de coincidir con la tienda.
Severidad / prioridad	Crítica / Alta
Casos que lo detectan	CP-04 (unitaria), CP-06 (integración) y CP-08 (sistema).
Evidencia esperada	Respuesta JSON de GET /api/productos/LAP-001 con "existencia": -1. CP-04, CP-06 y CP-08 fallidos en evidencias/regresion/DEF-01/junit.xml. Job «Unitarias» en rojo en GitHub Actions y el Pull Request bloqueado.
Evidencia obtenida	CP-04: Se esperaba InventarioException por existencia insuficiente. CP-06: Se esperaba que la venta se rechazara. CP-08: La venta de 11 unidades de LAP-001 (existencia 10) debía responder 422.
Responsable de corrección	Pablo Magaña (desarrollador del módulo de ventas). Verifica: Chuina García.

Defecto DEF-02: el reporte no marca para resurtir un producto que está justo en su mínimo

DEF-02 · comparación incorrecta del stock mínimo (prioridad media)

Descripción	El reporte solo marca para resurtir los productos por debajo de su stock mínimo y omite los que quedan exactamente en ese límite. La causa sería que Producto::bajoStockMinimo usa < en lugar de <=.
Requisito afectado	RF-05
Pasos para reproducir	1. Tener ACC-002 con existencia 12 y mínimo 5. 2. Vender 7 piezas (queda en 5). 3. Consultar GET /api/reporte.
Resultado esperado	ACC-002 aparece en la lista «bajoStockMinimo» (productos a resurtir).
Resultado obtenido	La lista no incluye ACC-002 ni RED-001 aunque los dos quedaron en su mínimo.
Impacto	La tienda no pide mercancía a tiempo y puede quedarse sin producto. No altera las existencias.
Severidad / prioridad	Moderada / Media
Casos que lo detectan	CP-09 (aceptación).
Evidencia esperada	Respuesta JSON de /api/reporte sin ACC-002 en «bajoStockMinimo». CP-09 fallido en evidencias/regresion/DEF-02/junit.xml. Job «Aceptación» en rojo en GitHub Actions.
Evidencia obtenida	CP-09: Productos a resurtir: se esperaban ACC-002 (5 de mínimo 5) y RED-001 (4 de mínimo 4).
Responsable de corrección	Pablo Magaña (desarrollador del módulo de reportes). Revalida: Ana Rodríguez (CP-09).

7. Repetición de pruebas después de corregir los defectos

7.1 Plan de re-pruebas

Defecto	Caso original	Acción de corrección propuesta	Prueba de repetición	Criterio de aprobación
DEF-01	CP-04 (y CP-06, CP-08)	Restaurar en Inventario::registrarSalida la comparación cantidad > existencia y lanzar InventarioException antes de modificar datos.	phpunit --filter con testVentaMayorALaExistencia (CP-04), testVentaRechazadaNoGuarda (CP-06) y testVentasInvalidas (CP-08); prueba manual con POST de 11 piezas.	Los tres casos aprobados, respuesta 422 y existencia sin cambios en 10.
DEF-02	CP-09	Cambiar en Producto::bajoStockMinimo la comparación < por <=.	phpunit --filter testReporteDeResurtido (CP-09); Ana revisa el reporte en /api/reporte.	CP-09 aprobado: ACC-002 y RED-001 aparecen por resurtir y MOU-010 no.

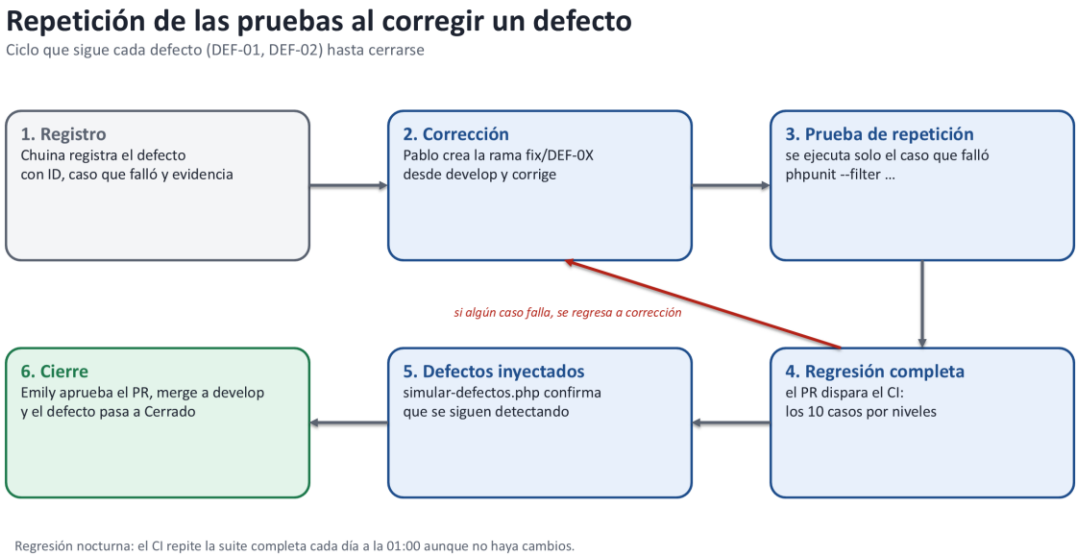


Figura 8. Ciclo para repetir las pruebas al corregir un defecto.

1. **Registro.** Chuina registra el defecto con su ID, el caso que falló, la evidencia y la prioridad.
2. **Corrección.** Pablo crea la rama fix/DEF-01 desde develop y corrige el código.
3. **Prueba de repetición.** Se ejecuta solo el caso que falló para confirmar la corrección.
4. **Regresión.** Al abrir el Pull Request, GitHub Actions ejecuta los diez casos y el job que inyecta DEF-01 y DEF-02.
5. **Cierre.** Con todo en verde, Emily aprueba el Pull Request y el defecto pasa a Cerrado.

7.2 Pruebas de regresión

Al corregir un defecto repetimos la suite completa, porque tarda menos de cinco segundos:

- CP-01 y CP-02, para verificar que se siguen registrando productos válidos y rechazando duplicados.
- CP-03 y CP-04, para asegurar el cálculo correcto de la existencia y el rechazo de ventas sin inventario.
- CP-05 y CP-06, para verificar la persistencia de los datos en el archivo.
- CP-07 y CP-08, para comprobar la API completa y sus validaciones.
- CP-09 y CP-10, para validar el proceso completo desde el punto de vista de la cliente.
- Job «Regresión DEF»: vuelve a inyectar DEF-01 y DEF-02 y falla si las pruebas ya no los detectan.
- Regresión programada: el CI repite la suite cada noche a la 01:00 aunque no haya cambios.

7.3 Registro posterior a la corrección

Actualizamos la bitácora con estos datos de la re-prueba:

Campo	DEF-01	DEF-02
ID del defecto	DEF-01	DEF-02
Versión corregida	Commit 185cafb (rama main)	Commit 185cafb (rama main)
Fecha de re-prueba	30 de octubre de 2026	30 de octubre de 2026
Responsable	Chuina García (QA)	Chuina García (QA) y Ana Rodríguez (cliente)
Resultado	Aprobado: CP-04, CP-06 y CP-08 pasan; regresión 10 de 10	Aprobado: CP-09 pasa; regresión 10 de 10
Evidencia	evidencias/regresion/DEF-01 y corregido (JUnit), Figuras 4 y 6, ejecución del CI	evidencias/regresion/DEF-02 y corregido (JUnit), Figura 4, ejecución del CI
Estado del defecto	Cerrado	Cerrado

8. Diseño del flujo de integración continua (CI)

Usamos Git, GitHub y GitHub Actions. El repositorio es público y el flujo ya se ejecutó con todos los jobs en verde:

- **Repositorio:** https://github.com/azharsonix/sistema-inventario-actividad-2
- **Ejecución del CI:** https://github.com/azharsonix/sistema-inventario-actividad-2/actions/runs/41169506352 (8 de 8 jobs exitosos)

8.1 Flujo de trabajo

Paso	Qué pasa	Cómo lo hacemos en el repositorio
1	El desarrollador modifica el código (implementa una función o corrige un defecto).	Trabaja en una rama feature/** o fix/** creada desde develop.
2	Commit y push al repositorio (se registra y comparte el cambio).	git commit y git push; abre un Pull Request hacia develop o main.
3	GitHub Actions inicia el flujo de CI (se prepara el entorno).	Máquina ubuntu-latest, PHP 8.2/8.3/8.4 y composer install con caché.
4	Se ejecutan las pruebas automatizadas.	Unitarias → Integración → Sistema → Aceptación → Regresión DEF, en cadena.
5A	Pruebas aprobadas: se permite la revisión y la integración.	Checks en verde; con una revisión aprobada se hace merge.
5B	Pruebas fallidas: se notifica, se revisan los logs y se corrige.	El PR queda bloqueado; GitHub avisa por correo y el JUnit queda como artefacto.

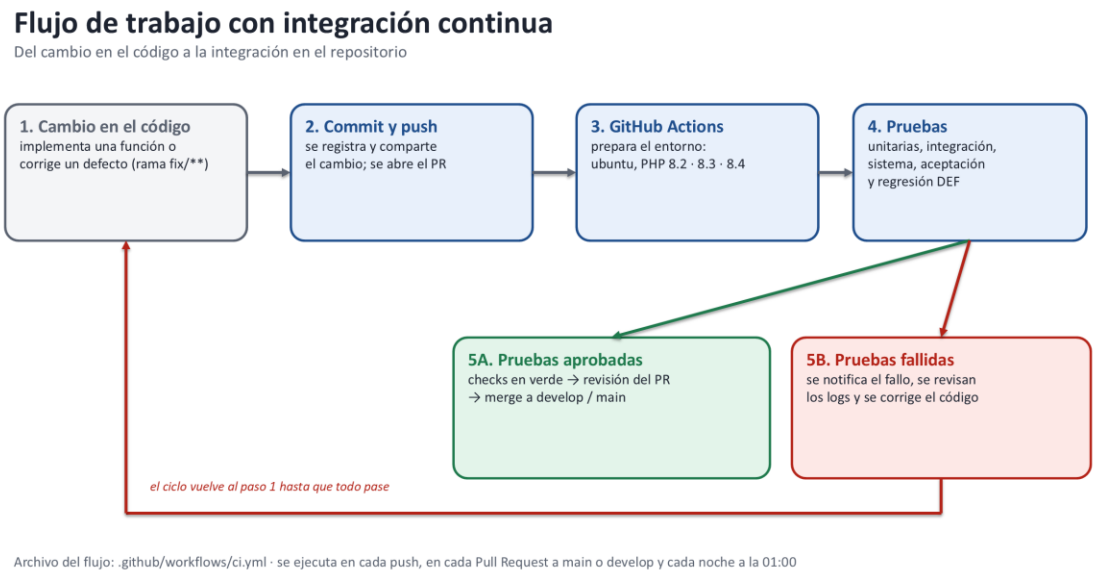


Figura 9. Flujo de trabajo: del cambio en el código a la integración (pasos 1 a 5A/5B).

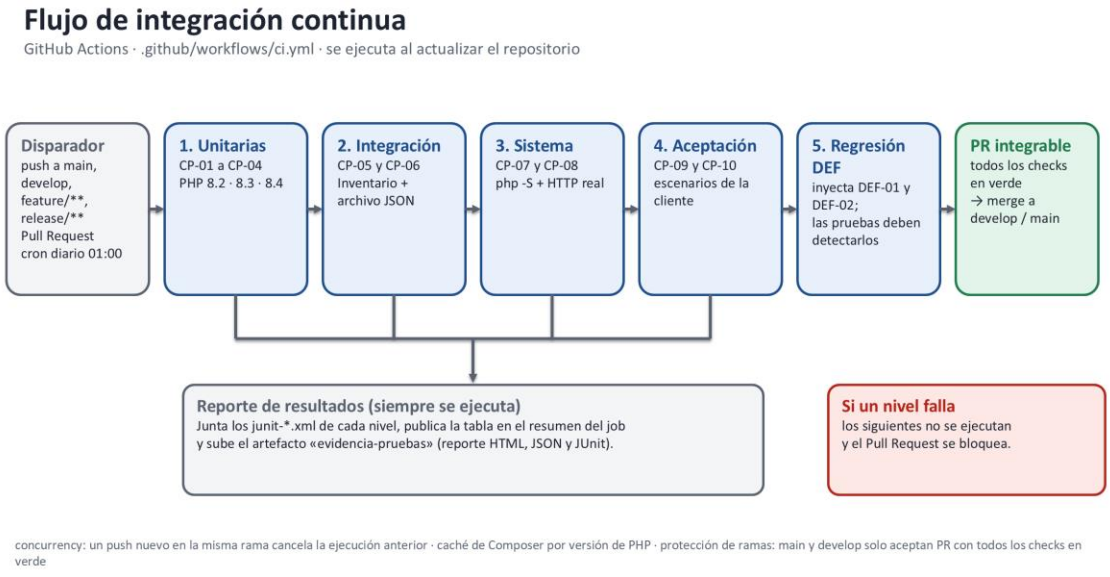


Figura 10. Jobs del pipeline y orden de ejecución.

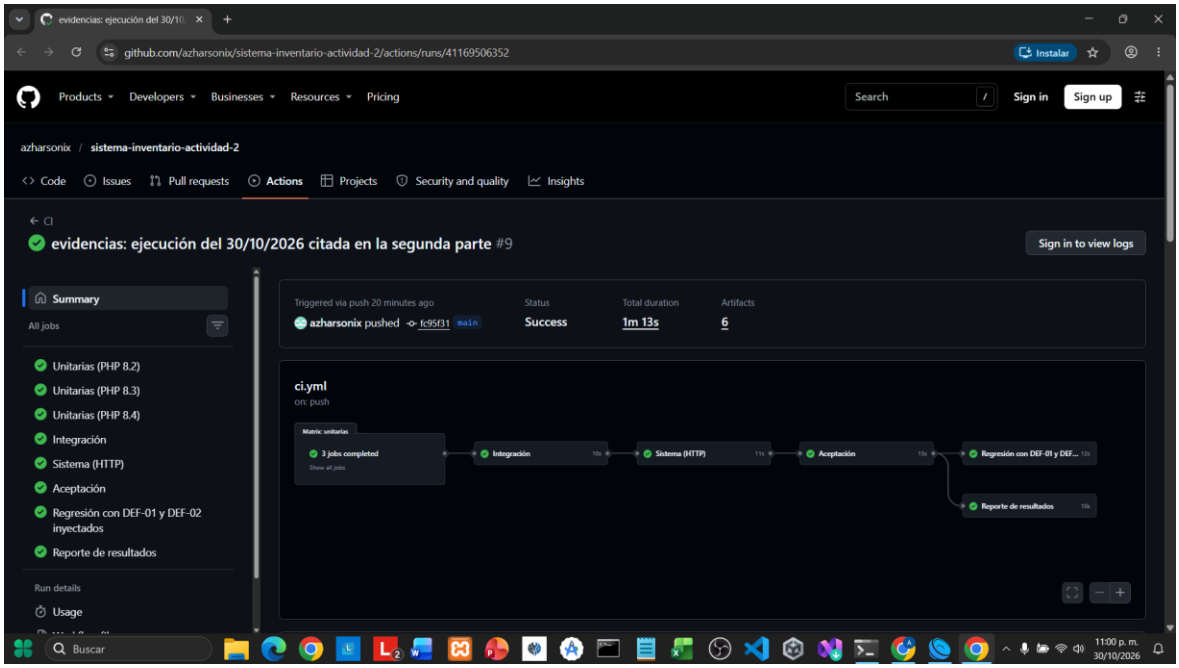


Figura 11. Ejecución del CI en GitHub Actions con los 8 jobs en verde.

8.2 Configuración de integración continua

El archivo está en .github/workflows/ci.yml del repositorio. Fragmento con los disparadores y el primer job:

```
name: CI

on:
  push:
    branches: [main, develop, "feature/**", "release/**", "hotfix/**"]
  pull_request:
    branches: [main, develop]
  schedule:
    - cron: "0 7 * * *" # 01:00 en Campeche (UTC-6)
  workflow_dispatch:

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: read

jobs:
  unitarias:
    name: Unitarias (PHP ${{ matrix.php }})
    runs-on: ubuntu-latest
    timeout-minutes: 10
    strategy:
      fail-fast: true
      matrix:
        php: ["8.2", "8.3", "8.4"]
    steps:
      - uses: actions/checkout@v4
      - uses: ./.github/actions/preparar-php
        with:
          php-version: ${{ matrix.php }}
```

```
- name: CP-01 a CP-04
  run: php scripts/ejecutar-pruebas.php --suite=Unitarias

- uses: actions/upload-artifact@v4
  if: always() && matrix.php == '8.2'
  with:
    name: junit-Unitarias
    path: evidencias/junit-Unitarias.xml
```

8.3 Pruebas que conviene automatizar

Prueba	Automatización	Frecuencia	Job del CI
CP-01 y CP-02: registro y duplicados	Sí	En cada push y Pull Request	Unitarias (3 versiones de PHP)
CP-03 y CP-04: ventas	Sí	En cada push y Pull Request	Unitarias (3 versiones de PHP)
CP-05 y CP-06: persistencia	Sí	En cada push y Pull Request	Integración
CP-07 y CP-08: API por HTTP	Sí	En cada push y Pull Request	Sistema (HTTP)
CP-09 y CP-10: escenarios de la cliente	Sí, con revisión de Ana antes de cada entrega	En cada Pull Request a main	Aceptación
Regresión con DEF-01 y DEF-02 inyectados	Sí	En cada Pull Request	Regresión DEF
Suite completa	Sí	Cada noche a la 01:00	Todos (cron)

8.4 Condiciones para aprobar la integración

Condición	Cómo se cumple
No integrar cambios si fallan las pruebas requeridas.	Protección de ramas en main y develop: los checks Unitarias, Integración, Sistema, Aceptación y Regresión deben estar en verde.
Revisión del código mediante Pull Request.	No se permite push directo; se pide una revisión aprobada (Emily).
Mantener un conjunto de pruebas de regresión.	Los diez casos más el job que inyecta DEF-01 y DEF-02.
Generar reportes y conservar los logs.	El job «Reporte» publica la tabla de resultados y sube el artefacto «evidencia-pruebas».
Datos ficticios y base de datos aislada.	Cada prueba usa su propio archivo JSON temporal; no toca data/inventario.json.
Pruebas adicionales antes de desplegar.	Antes de pasar a main se ejecutan los escenarios de aceptación y la regresión nocturna.

Conclusiones

Escribimos cinco requisitos con criterios verificables, y cada uno quedó cubierto por al menos dos casos. Las diez pruebas están automatizadas en cuatro niveles y la ronda 1 terminó con todas aprobadas.

Al inyectar los defectos comprobamos que las pruebas los detectan: DEF-01 provocó fallos en tres niveles distintos y DEF-02 en CP-09. La ronda 3 mostró que el porcentaje de aprobación debe leerse junto con la prioridad de lo que falla: un 90 % puede esconder un defecto que afecta a la tienda.

GitHub Actions ejecuta las pruebas en cada push y Pull Request, en tres versiones de PHP, y bloquea la integración de cualquier cambio que rompa una regla. Como siguiente mejora proponemos sumar al flujo las pruebas de la interfaz con Playwright.

Anexos y referencias

Anexo A. Cómo ejecutar las pruebas

Comando	Para qué
composer test	Ejecuta los 10 casos en la consola.
composer test:unitarias	Ejecuta un solo nivel (o test:integracion, test:sistema y test:aceptacion).
composer evidencia	Ejecuta las cuatro suites y genera la evidencia en evidencias/.
composer regresion	Inyecta DEF-01 y DEF-02 y comprueba que las pruebas los detectan.
php -S localhost:8000 -t public	Levanta la API del sistema (/api/productos, /api/reporte, /salud).

Anexo B. Evidencia en el repositorio

- evidencias/reporte-pruebas.html: reporte con totales, estado y tiempo de cada caso.
- evidencias/junit-*.xml: resultados de cada nivel en formato JUnit.
- evidencias/regresion/: JUnit y resumen de las rondas con DEF-01, DEF-02 y código corregido.

Referencias

- PHPUnit. (2025). PHPUnit 11.5 Manual. https://docs.phpunit.de/en/11.5/
- GitHub. (2026). Documentación de GitHub Actions. https://docs.github.com/es/actions
- ISTQB. (2026). Glosario de pruebas de software. https://glossary.istqb.org/
- Sommerville, I. (2011). Ingeniería de software (9.ª ed.). Pearson.